Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 11

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar cómo se comunican los componentes de React: datos del padre al hijo mediante props y avisos del hijo al padre mediante funciones recibidas por props.**
2. **Implementar la comunicación entre componentes hermanos elevando el estado al componente padre y calculando los datos derivados.**
3. **Utilizar useEffect para ejecutar código al montar, al actualizar y al desmontar un componente, y aplicar la comunicación entre componentes en el panel administrativo del proyecto integrador.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 11 (Comunicación entre componentes: props del padre al hijo, funciones recibidas por props, elevar el estado, estado derivado, ciclo de vida y useEffect) y el de la Semana 10 (componentes, props, useState, eventos y formularios controlados) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Node.js (versión LTS, que incluye npm), un editor de código (Visual Studio Code), un navegador web con sus herramientas de desarrollo y una cuenta de GitHub para el control de versiones.
- El proyecto de React creado en la Semana 9 (carpeta frontend) con los componentes de la Semana 10, y el panel administrativo del proyecto integrador que se construye con Django y React (un panel propio, no el Django Admin).

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (los componentes .jsx), de la página en el navegador y de la terminal cuando se ejecuten comandos.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

En la Semana 10 cada estudiante dividió su interfaz en componentes, pasó datos con props, guardó datos con useState y construyó formularios controlados. Sin embargo, cada componente guardaba su propio estado y los demás no se enteraban de lo que ocurría en él. En esta sesión los componentes aprenden a comunicarse: el padre entrega datos al hijo con props, el hijo avisa al

padre con funciones que recibe por props y, cuando dos componentes hermanos necesitan el mismo dato, el estado se eleva al componente padre.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante resuelve un ejercicio simple con los conceptos vistos en clase (un mini catálogo con carrito), dentro de la carpeta src/practica del proyecto frontend, para ver cada mecanismo funcionando en un caso pequeño. En la segunda parte, aplica esos mismos mecanismos en el panel administrativo de su proyecto integrador, cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda quién guarda cada dato y cómo viaja por la pantalla: los datos bajan por props y los avisos suben por funciones. Todavía no se consume la API (eso se hace en la Semana 12): los datos se escriben a mano con la misma forma que la respuesta de la API de la Semana 8. En clase la teoría se demuestra con una cafetería; cada estudiante la traslada a su panel administrativo con ayuda de la siguiente tabla de equivalencias.

Concepto trabajado en clase	**Equivalente en su panel administrativo**
Catálogo de productos con botón Agregar (en clase: Catalogo y ProductoCard)	El listado de su entidad principal, con un componente por fila y uno o más botones de acción por registro (por ejemplo, Eliminar o Activar).
Carrito, hermano del catálogo (en clase: el componente que muestra lo agregado y el total)	Un componente hermano del listado que muestre un resumen calculado con el mismo estado (total, conteo por estado o promedio).
Formulario que avisa al padre (en clase: ProductoForm con onGuardar)	El formulario de alta de su entidad principal, que envía el registro nuevo al componente padre del panel.
Buscador compartido (en clase: Buscador y Catalogo, con el estado en App)	Un filtro de texto o de estado, guardado en el padre, que controla qué registros muestra el listado.

Usuario (clic o escritura en la pantalla)

→ Componente hijo (ProductoCard, ProductoForm o Buscador)

→ Función recibida por props (onAgregar, onGuardar u onCambiar)

→ Componente padre (Practica o la pantalla del panel): setState

→ Estado elevado (carrito, registros o texto de búsqueda)

→ Props hacia los hijos (Catalogo, Carrito, Listado, Resumen)

→ React vuelve a dibujar la pantalla

Al finalizar el laboratorio, el estudiante deberá ser capaz de enviar datos del padre al hijo, enlazar eventos del hijo con funciones del padre, elevar el estado para compartirlo entre componentes hermanos, calcular datos derivados y usar useEffect con su limpieza, y de aplicar todo ello en el panel administrativo de su proyecto integrador y publicarlo en GitHub.

Enunciados

PARTE 1 — Comunicación entre componentes: un mini catálogo con carrito

Ejercicio 1 — Pasar datos del padre al hijo

Dentro de la carpeta frontend, cree la carpeta src/practica y, en ella, datos.js con cinco productos con id, nombre, precio (como texto, igual que la API) y stock: Café americano (6.50, stock 50), Capuchino (9.00, stock 40), Torta de chocolate (10.50, stock 5), Jugo de naranja (7.00, stock 0) y Tostada (4.50, stock 10). Cree los componentes Practica.jsx (padre), Catalogo.jsx y ProductoCard.jsx, de modo que Practica entregue los productos a Catalogo por props y Catalogo entregue cada producto a ProductoCard con map() y key. Muestre el nombre, el precio con Number(...).toFixed(2) y el mensaje Agotado cuando el stock sea 0. Documente el código y la página.

Respuesta:

Dentro de la carpeta frontend creé la carpeta src/practica con los datos de cinco insumos de la veterinaria: Jeringa 5ml, Vacuna antirrabica, Suero fisiologico, Antiparasitario y Gasa esteril, con el precio como texto igual que la API. El componente padre Practica entrega los insumos al Catalogo por props, y el Catalogo entrega cada insumo a su InsumoCard recorriendo la lista con map y una key basada en el id. Cada tarjeta muestra el nombre, el precio con dos decimales y el mensaje Agotado cuando el stock es cero (el Antiparasitario).

```
// practica/datos.js
// Insumos de la veterinaria con la forma de la API (precio como texto)
export const insumos = [
  { id: 1, nombre: 'Jeringa 5ml', precio: '1.50', stock: 100 },
  { id: 2, nombre: 'Vacuna antirrabica', precio: '25.00', stock: 12 },
  { id: 3, nombre: 'Suero fisiologico', precio: '8.00', stock: 1 },
  { id: 4, nombre: 'Antiparasitario', precio: '12.00', stock: 0 },
  { id: 5, nombre: 'Gasa esteril', precio: '2.50', stock: 40 },
]

// practica/Practica.jsx
import { useState } from 'react'
import { insumos } from './datos.js'
import Catalogo from './Catalogo.jsx'
import Consumo from './Consumo.jsx'
import Buscador from './Buscador.jsx'
import Aviso from '../components/Aviso.jsx'

// Componente padre: guarda el consumo, el texto de búsqueda y el aviso
function Practica() {
  const [consumo, setConsumo] = useState([])
  const [busqueda, setBusqueda] = useState('')
  const [aviso, setAviso] = useState('')

  const visibles = insumos.filter((i) =>
    i.nombre.toLowerCase().includes(busqueda.toLowerCase())
  )

  function agregarAlConsumo(insumo) {
    const existe = consumo.some((l) => l.id === insumo.id)
    if (existe) {
      setConsumo(
        consumo.map((l) => (l.id === insumo.id ? { ...l, cantidad: l.cantidad + 1 } : l))
      )
    } else {
      setConsumo([
        ...consumo,
        { id: insumo.id, nombre: insumo.nombre, precio: insumo.precio, cantidad: 1 },
      ])
    }
    setAviso(`Agregado: ${insumo.nombre}`)
  }

  function quitarDelConsumo(id) {
    setConsumo(consumo.filter((l) => l.id !== id))
  }

  function vaciarConsumo() {
    setConsumo([])
  }

  return (
    <div className="seccion-vet">
      <h2>Veterinaria: comunicación entre componentes</h2>
      <Buscador valor={busqueda} onCambiar={setBusqueda} />
      <Catalogo insumos={visibles} onAgregar={agregarAlConsumo} />
      <Consumo lineas={consumo} onQuitar={quitarDelConsumo} onVaciar={vaciarConsumo} />
      <Aviso mensaje={aviso} onTerminar={() => setAviso('')} />
    </div>
  )
}

export default Practica

// practica/Catalogo.jsx
import InsumoCard from './InsumoCard.jsx'

// Recibe la lista ya filtrada y entrega cada insumo a su tarjeta
function Catalogo({ insumos, onAgregar }) {
  if (insumos.length === 0) {
    return <p>No hay insumos que coincidan</p>
  }

  return (
    <ul>
      {insumos.map((insumo) => (
        <InsumoCard key={insumo.id} insumo={insumo} onAgregar={onAgregar} />
      ))}
    </ul>
  )
```



Ejercicio 2 — Reconocer los tipos de props

Complete una tabla con todas las props que pasó en el ejercicio anterior, con estas columnas: nombre de la prop, tipo de valor (texto, número, objeto, arreglo o función), qué componente la define y qué componente la lee. Explique con sus palabras por qué ProductoCard no puede modificar la prop producto y qué elemento de los Templates de Django (por ejemplo, el context de una View) cumple un papel parecido.

Respuesta:

Tabla con todas las props que pasé en el ejercicio anterior y en los siguientes:

Prop	Tipo de valor	La define	La lee
insumos	arreglo	Practica	Catalogo
insumo	objeto	Catalogo (uno por cada vuelta del map)	InsumoCard
onAgregar	función	Practica	Catalogo la reenvía e InsumoCard la ejecuta
lineas	arreglo	Practica	Consumo
onQuitar y onVaciar	función	Practica	Consumo
valor	texto	Practica	Buscador
onCambiar	función	Practica	Buscador
mensaje	texto	Practica	Aviso
onTerminar	función	Practica	Aviso

InsumoCard no puede modificar la prop insumo porque es un dato que pertenece a su padre: la tarjeta solo recibe una copia para mostrarla. Si la tarjeta lo cambiara, la pantalla no se volvería a dibujar y el padre seguiría con el dato original. Los cambios se piden al padre con una función. En los Templates de Django

algo parecido es el context que una View le entrega al Template: el template lee esos datos pero no los modifica; quien los cambia es la View.

Ejercicio 3 — Enlazar un evento con una función del padre

Agregue a ProductoCard un botón Agregar (desactivado cuando el stock sea 0) que ejecute la función onAgregar recibida por props, enviándole el producto. En Practica defina la función agregarAlCarrito, que por ahora solo muestre con console.log el nombre del producto, y pásela hasta ProductoCard. Compruebe en la consola del navegador. Después escriba temporalmente onClick={onAgregar(producto)}, documente el error que aparece y explique por qué ocurre y cómo se corrige.

Respuesta:

Agregué a InsumoCard un botón Agregar, desactivado cuando el stock es cero, que ejecuta la función onAgregar recibida por props y le envía el insumo. En Practica la función agregarAlConsumo baja hasta la tarjeta pasando por el Catalogo. En esta entrega la función ya actualiza el consumo, que es lo que se pide en el ejercicio siguiente. Sobre escribir onClick con el insumo ya entre paréntesis: eso ejecuta la función en el momento de dibujar la tarjeta y no cuando se hace clic, y como la función cambia el estado, React tendría que dibujar otra vez y volver a ejecutarla sin parar, hasta mostrar el error de demasiadas redibujadas. Se corrige pasando una función flecha que llama a onAgregar solo cuando ocurre el clic.

```
// practica/InsumoCard.jsx
// Muestra un insumo y avisa al padre cuando se pulsa Agregar
function InsumoCard({ insumo, onAgregar }) {
  return (
    <li>
      {insumo.nombre} — S/ {Number(insumo.precio).toFixed(2)} —{' '}
      {insumo.stock === 0 ? (
        <span className="sin-stock">Agotado</span>
      ) : (
        `Stock: ${insumo.stock}`
      )}{' '}
      <button onClick={() => onAgregar(insumo)} disabled={insumo.stock === 0}>
        Agregar
      </button>
    </li>
  )
}

export default InsumoCard
```



Ejercicio 4 — Elevar el estado: un carrito compartido

Guarde el carrito en el componente Practica con useState, como un arreglo de líneas con id, nombre, precio y cantidad. Complete agregarAlCarrito para que, si el producto ya está en el carrito, aumente su cantidad con map(), y si no está, agregue una línea nueva con un arreglo nuevo (sin push). Cree el componente Carrito, hermano de Catalogo, que reciba las líneas por props y las muestre. Explique con sus palabras por qué el carrito no puede guardarse dentro de ProductoCard ni dentro de Catalogo.

Respuesta:

El consumo de la cita se guarda en el componente Practica con useState, como un arreglo de líneas con id, nombre, precio y cantidad. Al agregar, si el insumo ya está en el consumo se aumenta su cantidad con map, y si no está se crea un arreglo nuevo con la línea nueva, sin usar push. El componente Consumo es hermano del Catalogo y recibe las líneas por props. El consumo no puede guardarse dentro de InsumoCard ni dentro del Catalogo porque esos componentes no se ven entre sí: el Consumo necesita leer lo que el Catalogo agrega, y el único lugar común que ve a los dos es el padre, por eso el estado vive ahí.

```jsx
// practica/Practica.jsx
import { useState } from 'react'
import { insumos } from './datos.js'
import Catalogo from './Catalogo.jsx'
import Consumo from './Consumo.jsx'
import Buscador from './Buscador.jsx'
import Aviso from '../components/Aviso.jsx'

// Componente padre: guarda el consumo, el texto de búsqueda y el aviso
function Practica() {
  const [consumo, setConsumo] = useState([])
  const [busqueda, setBusqueda] = useState('')
  const [aviso, setAviso] = useState('')

  const visibles = insumos.filter((i) =>
    i.nombre.toLowerCase().includes(busqueda.toLowerCase())
  )

  function agregarAlConsumo(insumo) {
    const existe = consumo.some((l) => l.id === insumo.id)
    if (existe) {
      setConsumo(
        consumo.map((l) => (l.id === insumo.id ? { ...l, cantidad: l.cantidad + 1 } : l))
      )
    } else {
      setConsumo([
        ...consumo,
        { id: insumo.id, nombre: insumo.nombre, precio: insumo.precio, cantidad: 1 },
      ])
    }
    setAviso(`Agregado: ${insumo.nombre}`)
  }

  function quitarDelConsumo(id) {
    setConsumo(consumo.filter((l) => l.id !== id))
  }

  function vaciarConsumo() {
    setConsumo([])
  }

  return (
    <div className="seccion-vet">
      <h2>Veterinaria: comunicación entre componentes</h2>
      <Buscador valor={busqueda} onCambiar={setBusqueda} />
      <Catalogo insumos={visibles} onAgregar={agregarAlConsumo} />
      <Consumo lineas={consumo} onQuitar={quitarDelConsumo} onVaciar={vaciarConsumo} />
      <Aviso mensaje={aviso} onTerminar={() => setAviso('')} />
    </div>
  )
}

export default Practica

// practica/Consumo.jsx
import { useEffect } from 'react'

// Muestra los insumos usados en la cita y avisa al padre al quitar o vaciar
function Consumo({ lineas, onQuitar, onVaciar }) {
  // Datos derivados: se calculan en cada dibujo, no necesitan su propio estado
  const total = lineas.reduce((suma, l) => suma + Number(l.precio) * l.cantidad, 0)
  const unidades = lineas.reduce((suma, l) => suma + l.cantidad, 0)

  useEffect(() => {
    document.title = `Consumo (${unidades}) · Veterinaria`
    console.log('Efecto del consumo: unidades =', unidades)
  }, [unidades])

  return (
    <div>
      <h3>Consumo de la cita</h3>
      {lineas.length === 0 && <p>El consumo está vacío</p>}
      <ul>
        {lineas.map((l) => (
          <li key={l.id}>
            {l.nombre} x {l.cantidad} — S/ {(Number(l.precio) * l.cantidad).toFixed(2)}{' '}
            <button onClick={() => onQuitar(l.id)}>Quitar</button>
          </li>
        ))}
      </ul>
```



Ejercicio 5 — Calcular datos derivados

En Carrito calcule con reduce() el total a pagar y la cantidad de unidades, sin crear estados nuevos para ellos, y muéstrelos (por ejemplo, Total: S/ 24.50). Pruebe agregando dos Capuchinos y un Café americano y documente el resultado. Explique con sus palabras por qué el total no necesita su propio useState.

Respuesta:

En Consumo calculo con reduce el total a pagar y la cantidad de unidades, sin crear estados nuevos. Prueba: agregué dos Vacunas antirrábicas y una Jeringa. Resultado: 3 unidades y un total de S/ 51.50 (dos por 25.00 más 1.50). El total no necesita su propio estado porque se puede calcular siempre a partir de las líneas; si lo guardara aparte tendría que acordarme de actualizarlo en cada cambio y podría quedar distinto del consumo real.



Ejercicio 6 — Quitar y vaciar desde el hijo

Agregue a cada línea del carrito un botón Quitar que avise al padre con onQuitar(id), y un botón Vaciar carrito con onVaciar. El padre debe actualizar el estado con filter() o con un arreglo vacío, y Carrito debe mostrar El carrito está vacío con && cuando no haya líneas. Verifique que en ninguna parte se usa push ni splice sobre el estado.

Respuesta:

Cada línea del consumo tiene un botón Quitar que avisa al padre con onQuitar y le envía el id, y hay un botón Vaciar consumo con onVaciar. El padre actualiza el estado con filter en un caso y con un arreglo vacío en el otro. Cuando no hay líneas el Consumo muestra El consumo está vacío usando el operador &&. En ninguna parte se usa push ni splice sobre el estado. Primero quité la Vacuna (queda solo la Jeringa) y luego vacié todo.

```jsx
// practica/Practica.jsx
import { useState } from 'react'
import { insumos } from './datos.js'
import Catalogo from './Catalogo.jsx'
import Consumo from './Consumo.jsx'
import Buscador from './Buscador.jsx'
import Aviso from '../components/Aviso.jsx'

// Componente padre: guarda el consumo, el texto de búsqueda y el aviso
function Practica() {
  const [consumo, setConsumo] = useState([])
  const [busqueda, setBusqueda] = useState('')
  const [aviso, setAviso] = useState('')

  const visibles = insumos.filter((i) =>
    i.nombre.toLowerCase().includes(busqueda.toLowerCase())
  )

  function agregarAlConsumo(insumo) {
    const existe = consumo.some((l) => l.id === insumo.id)
    if (existe) {
      setConsumo(
        consumo.map((l) => (l.id === insumo.id ? { ...l, cantidad: l.cantidad + 1 } : l))
      )
    } else {
      setConsumo([
        ...consumo,
        { id: insumo.id, nombre: insumo.nombre, precio: insumo.precio, cantidad: 1 },
      ])
    }
    setAviso(`Agregado: ${insumo.nombre}`)
  }

  function quitarDelConsumo(id) {
    setConsumo(consumo.filter((l) => l.id !== id))
  }

  function vaciarConsumo() {
    setConsumo([])
  }

  return (
    <div className="seccion-vet">
      <h2>Veterinaria: comunicación entre componentes</h2>
      <Buscador valor={busqueda} onCambiar={setBusqueda} />
      <Catalogo insumos={visibles} onAgregar={agregarAlConsumo} />
      <Consumo lineas={consumo} onQuitar={quitarDelConsumo} onVaciar={vaciarConsumo} />
      <Aviso mensaje={aviso} onTerminar={() => setAviso('')} />
    </div>
  )
}

export default Practica
```

Inicio | Práctica: insumos | Panel de citas

Veterinaria: comunicación entre componentes

Buscar insumo

- Jeringa 5ml — S/ 1.50 — Stock: 100 Agregar
- Vacuna antirrabica — S/ 25.00 — Stock: 12 Agregar
- Suero fisiologico — S/ 8.00 — Stock: 1 Agregar
- Antiparasitario — S/ 12.00 — **Agotado** Agregar
- Gasa esteril — S/ 2.50 — Stock: 40 Agregar

Consumo de la cita

- Jeringa 5ml x 1 — S/ 1.50 Quitar

Unidades: 1

Total: S/ 1.50

Vaciar consumo

Agregado: Jeringa 5ml

Inicio | Práctica: insumos | Panel de citas

Veterinaria: comunicación entre componentes

Buscar insumo

- Jeringa 5ml — S/ 1.50 — Stock: 100 Agregar
- Vacuna antirrabica — S/ 25.00 — Stock: 12 Agregar
- Suero fisiologico — S/ 8.00 — Stock: 1 Agregar
- Antiparasitario — S/ 12.00 — **Agotado** Agregar
- Gasa esteril — S/ 2.50 — Stock: 40 Agregar

Consumo de la cita

El consumo está vacío

Unidades: 0

Total: S/ 0.00

Vaciar consumo

Agregado: Jeringa 5ml

Ejercicio 7 — Hermanos que comparten un filtro

Cree el componente Buscador, que reciba valor y onCambiar por props y muestre un campo de texto controlado, y guarde el texto en el estado de Practica. Filtre los productos con filter() y includes() antes de entregarlos a Catalogo, y muestre No hay productos que coincidan cuando no haya resultados. Documente la lista al escribir ca y al escribir zzz. Explique por qué Buscador y Catalogo no se comunican directamente.

Respuesta:

Creé el componente Buscador, que recibe valor y onCambiar por props y muestra un campo de texto controlado; el texto se guarda en el estado de Practica. El padre filtra los insumos con filter e includes antes de entregarlos al Catalogo, y el Catalogo muestra No hay insumos que coincidan cuando la lista llega vacía. Al escribir ca aparece solo la Vacuna antirrabica; al escribir zzz aparece el mensaje. Buscador y Catalogo no se comunican directamente porque son hermanos: el Buscador avisa al padre y el padre decide qué lista le entrega al Catalogo.

```jsx
// practica/Buscador.jsx
// Campo de texto controlado: el valor vive en el padre
function Buscador({ valor, onCambiar }) {
  return (
    <input
      type="text"
      placeholder="Buscar insumo"
      value={valor}
      onChange={(e) => onCambiar(e.target.value)}
    />
  )
}

export default Buscador
```





Ejercicio 8 — Ciclo de vida con useEffect y verificación

En Carrito, use useEffect con la dependencia de la cantidad de unidades para cambiar document.title (por ejemplo, Carrito (3) — Cafetería). Cree además un componente Aviso que muestre Agregado: <nombre> y desaparezca a los 3 segundos con setTimeout, limpiando el temporizador con la función que devuelve el efecto. Agregue un console.log dentro del efecto y explique qué ocurre con [], con [cantidad] y sin arreglo de dependencias, y por qué en desarrollo el mensaje puede aparecer dos veces (React.StrictMode). Ejecute npm run lint y npm run build y documente la salida.

Respuesta:

En Consumo uso un efecto cuya dependencia es la cantidad de unidades para cambiar el título de la pestaña (por ejemplo, Consumo (3) · Veterinaria). El componente Aviso muestra el mensaje Agregado y el nombre del insumo, y desaparece a los 3 segundos con un temporizador; el efecto devuelve una función de limpieza que cancela el temporizador. Además puse un mensaje en consola dentro del efecto. Con un arreglo de dependencias vacío el efecto corre una sola vez al montar el componente; con la cantidad como dependencia corre al montar y cada vez que cambia esa cantidad; sin arreglo de dependencias corre después de cada dibujo. En desarrollo el mensaje puede aparecer dos veces porque React.StrictMode monta, desmonta y vuelve a montar el componente para detectar efectos mal limpiados. Ejecuté npm run lint sin errores y npm run build terminó correctamente.

```jsx
// practica/Consumo.jsx
import { useEffect } from 'react'

// Muestra los insumos usados en la cita y avisa al padre al quitar o vaciar
function Consumo({ lineas, onQuitar, onVaciar }) {
  // Datos derivados: se calculan en cada dibujo, no necesitan su propio estado
  const total = lineas.reduce((suma, l) => suma + Number(l.precio) * l.cantidad, 0)
  const unidades = lineas.reduce((suma, l) => suma + l.cantidad, 0)

  useEffect(() => {
    document.title = `Consumo (${unidades}) · Veterinaria`
    console.log('Efecto del consumo: unidades =', unidades)
  }, [unidades])

  return (
    <div>
      <h3>Consumo de la cita</h3>
      {lineas.length === 0 && <p>El consumo está vacío</p>}
      <ul>
        {lineas.map((l) => (
          <li key={l.id}>
            {l.nombre} x {l.cantidad} — S/ {(Number(l.precio) * l.cantidad).toFixed(2)}{' '}
            <button onClick={() => onQuitar(l.id)}>Quitar</button>
          </li>
        ))}
      </ul>
      <p>Unidades: {unidades}</p>
      <p>Total: S/ {total.toFixed(2)}</p>
      <button onClick={onVaciar}>Vaciar consumo</button>
    </div>
  )
}

export default Consumo

// components/Aviso.jsx
import { useEffect } from 'react'

// Mensaje que desaparece solo a los 3 segundos
function Aviso({ mensaje, onTerminar }) {
  useEffect(() => {
    if (mensaje === '') return
    const temporizador = setTimeout(onTerminar, 3000)
    // Limpieza: cancela el temporizador si el componente se desmonta o cambia el mensaje
    return () => clearTimeout(temporizador)
  }, [mensaje, onTerminar])

  if (mensaje === '') return null
  return <p className="aviso">{mensaje}</p>
}

export default Aviso
```

Inicio | Práctica: insumos | Panel de citas

Veterinaria: comunicación entre componentes

Buscar insumo

- Jeringa 5ml — S/ 1.50 — Stock: 100 Agregar
- Vacuna antirrabica — S/ 25.00 — Stock: 12 Agregar
- Suero fisiologico — S/ 8.00 — Stock: 1 Agregar
- Antiparasitario — S/ 12.00 — **Agotado** Agregar
- Gasa esteril — S/ 2.50 — Stock: 40 Agregar

Consumo de la cita

- Suero fisiologico x 1 — S/ 8.00 Quitar

Unidades: 1

Total: S/ 8.00

Vaciar consumo

Agregado: Suero fisiologico

Inicio | Práctica: insumos | Panel de citas

Veterinaria: comunicación entre componentes

Buscar insumo

- Jeringa 5ml — S/ 1.50 — Stock: 100 Agregar
- Vacuna antirrabica — S/ 25.00 — Stock: 12 Agregar
- Suero fisiologico — S/ 8.00 — Stock: 1 Agregar
- Antiparasitario — S/ 12.00 — **Agotado** Agregar
- Gasa esteril — S/ 2.50 — Stock: 40 Agregar

Consumo de la cita

- Suero fisiologico x 1 — S/ 8.00 Quitar

Unidades: 1

Total: S/ 8.00

Vaciar consumo

```
Consola del navegador y título de la pestaña
[log] Efecto del consumo: unidades = 0
[log] Efecto del consumo: unidades = 0
[log] Efecto del consumo: unidades = 1
[log] Efecto del consumo: unidades = 2
[log] Efecto del consumo: unidades = 3
[log] Efecto del consumo: unidades = 1
[log] Efecto del consumo: unidades = 0
[log] Efecto del consumo: unidades = 1
Título de la pestaña: Consumo (1) · Veterinaria
```

```
PS ...\frontend> npm run lint

> frontend@0.0.0 lint
> eslint .
```

```
PS ...\frontend> npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.5 building client environment for production...
transforming...
✓ 43 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  0.47 kB │ gzip:  0.30 kB
dist/assets/index-xrUpEBpY.css   1.27 kB │ gzip:  0.47 kB
dist/assets/index-yf61NHLG.js  236.51 kB │ gzip: 73.25 kB

✓ built in 274ms
```

PARTE 2 — Comunicación entre componentes en el panel administrativo del proyecto integrador

A partir de aquí trabajará en el panel administrativo de su proyecto integrador. El objetivo es construir una pantalla de gestión de su entidad principal aplicando lo practicado en la Parte 1: un listado cuyas filas avisan al padre, un formulario de alta que envía el registro nuevo al padre, un resumen que depende del mismo estado, un buscador compartido y un efecto con useEffect. Los datos se escriben a mano con la forma de la respuesta de la API de la Semana 8, y debe cumplir obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Identificar los componentes del panel

Complete la tabla de equivalencias con su proyecto: indique cuál es su entidad principal, qué componente será el padre que guarde su estado, cuál será el listado y su fila, cuál será el formulario de alta, cuál será el resumen hermano del listado (con qué dato derivado) y qué campo servirá para el buscador. Incluya un esquema (a mano, en texto o con una captura) del árbol de componentes del panel, indicando con flechas qué datos bajan por props y qué avisos suben por funciones.

Respuesta:

Mi entidad principal es la cita de la veterinaria. Tabla de equivalencias de mi panel:

Concepto	En mi panel de la veterinaria
Componente padre que guarda el estado	PanelCitas
Listado y fila	CitaListado y CitaFila (botones Confirmar y Eliminar)
Formulario de alta	CitaForm, entrega la cita nueva con onGuardar
Resumen hermano del listado	ResumenCitas: total de citas y conteo por estado (pendientes, confirmadas y atendidas)
Campo del buscador	La mascota (BuscadorCitas)

Árbol de componentes: PanelCitas (estado) contiene a CitaForm, BuscadorCitas, CitaListado (con una CitaFila por cita), ResumenCitas y Aviso. Bajan por props: las citas visibles hacia el listado y las filas, todas las citas hacia el resumen, el texto de búsqueda hacia el buscador y el mensaje hacia el aviso. Suben por funciones: onGuardar desde el formulario, onConfirmar y onEliminar desde cada fila, onCambiar desde el buscador y onTerminar desde el aviso.

```
PS ...\frontend> dir src
src/App.css
src/App.jsx
src/Vet.jsx
src/assets/hero.png
src/assets/react.svg
src/assets/vite.svg
src/components/Aviso.jsx
src/components/BuscadorCitas.jsx
src/components/CitaCard.jsx
src/components/CitaFila.jsx
src/components/CitaForm.jsx
src/components/CitaListado.jsx
src/components/ConsumoItem.jsx
src/components/InsumoCard.jsx
src/components/InsumoLista.jsx
src/components/LibroCard.jsx
src/components/LibroLista.jsx
src/components/Panel.jsx
src/components/PanelCitas.jsx
src/components/PrestamoItem.jsx
src/components/ResumenCitas.jsx
src/components/SocioCard.jsx
src/components/Tarjeta.jsx
src/datos.js
src/datosBiblioteca.js
src/index.css
src/main.jsx
src/practica/Buscador.jsx
src/practica/Catalogo.jsx
src/practica/Consumo.jsx
src/practica/InsumoCard.jsx
src/practica/Practica.jsx
src/practica/datos.js
```

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. El proyecto frontend se ejecuta en http://localhost:5173/ y los ejercicios de la Parte 1 están en src/practica.	Captura de la terminal, del navegador y de la estructura de carpetas.
2. El panel está dividido en componentes, cada uno en su archivo de src/components (al menos listado, fila, formulario y resumen).	Captura de la estructura de carpetas.
3. El estado de la entidad vive en un único componente padre y se entrega a los hijos por props.	Código del padre + captura.
4. Cada fila tiene botones de acción que avisan al padre con funciones recibidas por props.	Código + captura antes y después de la acción.
5. El formulario de alta es un componente hijo controlado, valida antes de enviar y entrega el registro al padre con onGuardar (o equivalente).	Código + captura de un alta válida y una inválida.
6. El estado siempre se actualiza con un arreglo nuevo (spread, map o filter); no hay push ni splice sobre el estado.	Código de las funciones del padre.
7. Un componente hermano del listado muestra un	Código + captura.

resumen con datos derivados calculados sin useState.	
8. Un buscador o filtro controlado por el padre actúa sobre el listado y muestra un mensaje cuando no hay resultados.	Código + captura de dos búsquedas.
9. Hay al menos un useEffect con arreglo de dependencias (y limpieza si usa un temporizador); la consola no muestra advertencias y npm run lint y npm run build terminan sin errores.	Código + capturas de la consola y de la terminal.

Ejercicio 10 — Estado elevado y listado del panel

Cree el componente padre de la pantalla (por ejemplo, PanelProductos.jsx) con un arreglo de al menos cinco registros de su entidad principal en useState, con la forma de la respuesta de la API de la Semana 8 (los decimales como texto). Entregue los registros al listado por props y dibuje cada fila con un componente propio, con map() y key basada en el id. Cada componente debe estar en su archivo dentro de src/components.

Respuesta:

El componente padre PanelCitas guarda en useState un arreglo de seis citas con la forma de la respuesta de la API de la Semana 8 (mascota, dueño, servicio, hora y estado). Entrega las citas al listado por props y cada fila se dibuja con su propio componente, con map y key basada en el id. Cada componente está en su archivo dentro de src/components.

```jsx
// components/PanelCitas.jsx
import { useEffect, useState } from 'react'
import { citasPanel } from '../datos.js'
import CitaListado from './CitaListado.jsx'
import CitaForm from './CitaForm.jsx'
import ResumenCitas from './ResumenCitas.jsx'
import BuscadorCitas from './BuscadorCitas.jsx'
import Aviso from './Aviso.jsx'

// Componente padre del panel: aquí vive el estado de las citas
function PanelCitas() {
  const [citas, setCitas] = useState(citasPanel)
  const [busqueda, setBusqueda] = useState('')
  const [aviso, setAviso] = useState('')

  const visibles = citas.filter((c) =>
    c.mascota.toLowerCase().includes(busqueda.toLowerCase())
  )

  // El título de la pestaña sigue la cantidad de citas
  useEffect(() => {
    document.title = `Panel de citas (${citas.length})`
  }, [citas.length])

  function guardarCita(datos) {
    const nueva = { id: Math.max(0, ...citas.map((c) => c.id)) + 1, ...datos, estado: 'Pendiente' }
    setCitas([...citas, nueva])
    setAviso(`Cita guardada: ${nueva.mascota}`)
  }

  function eliminarCita(id) {
    setCitas(citas.filter((c) => c.id !== id))
  }

  function confirmarCita(id) {
    setCitas(citas.map((c) => (c.id === id ? { ...c, estado: 'Confirmada' } : c)))
  }

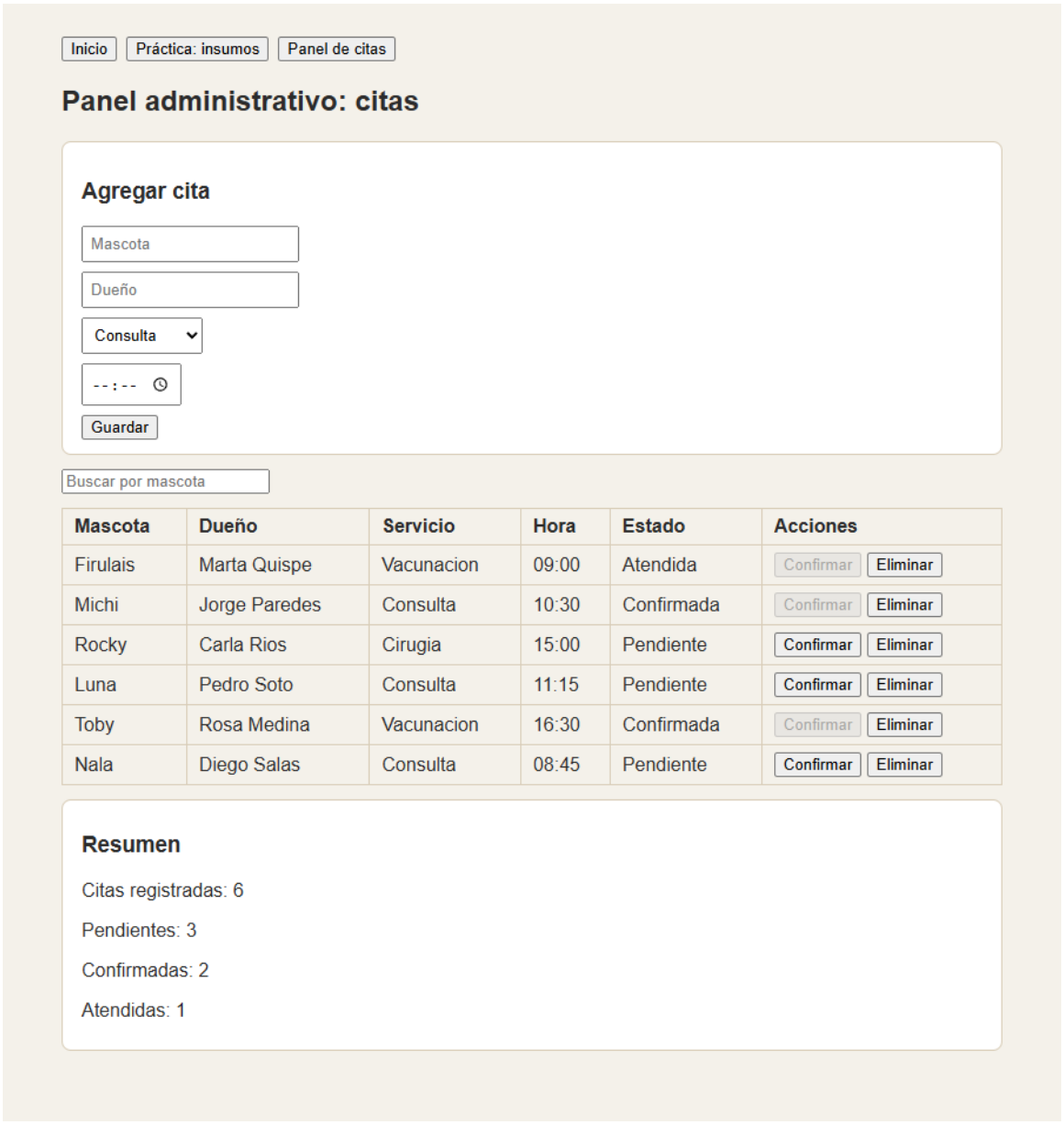
  return (
    <div className="seccion-vet">
      <h2>Panel administrativo: citas</h2>
      <CitaForm onGuardar={guardarCita} />
      <Aviso mensaje={aviso} onTerminar={() => setAviso('')} />
      <BuscadorCitas valor={busqueda} onCambiar={setBusqueda} />
      <CitaListado citas={visibles} onConfirmar={confirmarCita} onEliminar={eliminarCita} />
      <ResumenCitas citas={citas} />
    </div>
  )
}

export default PanelCitas

// components/CitaListado.jsx
import CitaFila from './CitaFila.jsx'

// Tabla de citas: recibe la lista por props y entrega cada cita a su fila
function CitaListado({ citas, onConfirmar, onEliminar }) {
  if (citas.length === 0) {
    return <p>No hay citas que coincidan</p>
  }

  return (
    <table className="tabla-libros">
      <thead>
        <tr>
          <th>Mascota</th>
          <th>Dueño</th>
          <th>Servicio</th>
          <th>Hora</th>
          <th>Estado</th>
          <th>Acciones</th>
        </tr>
      </thead>
      <tbody>
        {citas.map((cita) => (
          <CitaFila key={cita.id} cita={cita} onConfirmar={onConfirmar} onEliminar={onEliminar} />
        ))}
      </tbody>
    </table>
  )
```



Ejercicio 11 — Eventos enlazados en el listado

Agregue a cada fila al menos dos botones de acción de su panel (por ejemplo, Eliminar y Activar o Desactivar) que avisen al padre con funciones recibidas por props. El padre debe actualizar el estado con filter() o map(), nunca con push ni splice. Documente la lista antes y después de cada acción y explique por qué la fila no modifica la lista por sí misma.

Respuesta:

Cada fila tiene dos botones, Confirmar y Eliminar, que avisan al padre con funciones recibidas por props. Confirmar queda desactivado si la cita ya no está pendiente. El padre actualiza el estado con map para confirmar y con filter para eliminar, nunca con push ni splice. Antes: seis citas, con Rocky pendiente. Después de confirmar a Rocky su estado pasó a Confirmada y se desactivó su botón. Después de eliminar a Nala quedaron cinco citas. La fila no modifica la lista por sí sola porque solo recibe su cita como dato de lectura; la lista pertenece al padre y por eso la fila le avisa y es el padre quien cambia su estado.

```jsx
// components/CitaFila.jsx
// Una fila del listado: muestra la cita y avisa al padre con las funciones recibidas
function CitaFila({ cita, onConfirmar, onEliminar }) {
  return (
    <tr>
      <td>{cita.mascota}</td>
      <td>{cita.dueno}</td>
      <td>{cita.servicio}</td>
      <td>{cita.hora}</td>
      <td>{cita.estado}</td>
      <td>
        <button onClick={() => onConfirmar(cita.id)} disabled={cita.estado !== 'Pendiente'}>
          Confirmar
        </button>{' '}
        <button onClick={() => onEliminar(cita.id)}>Eliminar</button>
      </td>
    </tr>
  )
}

export default CitaFila

// components/PanelCitas.jsx
import { useEffect, useState } from 'react'
import { citasPanel } from '../datos.js'
import CitaListado from './CitaListado.jsx'
import CitaForm from './CitaForm.jsx'
import ResumenCitas from './ResumenCitas.jsx'
import BuscadorCitas from './BuscadorCitas.jsx'
import Aviso from './Aviso.jsx'

// Componente padre del panel: aquí vive el estado de las citas
function PanelCitas() {
  const [citas, setCitas] = useState(citasPanel)
  const [busqueda, setBusqueda] = useState('')
  const [aviso, setAviso] = useState('')

  const visibles = citas.filter((c) =>
    c.mascota.toLowerCase().includes(busqueda.toLowerCase())
  )

  // El título de la pestaña sigue la cantidad de citas
  useEffect(() => {
    document.title = `Panel de citas (${citas.length})`
  }, [citas.length])

  function guardarCita(datos) {
    const nueva = { id: Math.max(0, ...citas.map((c) => c.id)) + 1, ...datos, estado: 'Pendiente' }
    setCitas([...citas, nueva])
    setAviso(`Cita guardada: ${nueva.mascota}`)
  }

  function eliminarCita(id) {
    setCitas(citas.filter((c) => c.id !== id))
  }

  function confirmarCita(id) {
    setCitas(citas.map((c) => (c.id === id ? { ...c, estado: 'Confirmada' } : c)))
  }

  return (
    <div className="seccion-vet">
      <h2>Panel administrativo: citas</h2>
      <CitaForm onGuardar={guardarCita} />
      <Aviso mensaje={aviso} onTerminar={() => setAviso('')} />
      <BuscadorCitas valor={busqueda} onCambiar={setBusqueda} />
      <CitaListado citas={visibles} onConfirmar={confirmarCita} onEliminar={eliminarCita} />
      <ResumenCitas citas={citas} />
    </div>
  )
}

export default PanelCitas
```

Inicio | Práctica: insumos | Panel de citas

Panel administrativo: citas

Agregar cita

Mascota

Dueño

Consulta

--:--

Guardar

Buscar por mascota

Mascota	Dueño	Servicio	Hora	Estado	Acciones
Firulais	Marta Quispe	Vacunacion	09:00	Atendida	Confirmar Eliminar
Michi	Jorge Paredes	Consulta	10:30	Confirmada	Confirmar Eliminar
Rocky	Carla Rios	Cirugia	15:00	Pendiente	Confirmar Eliminar
Luna	Pedro Soto	Consulta	11:15	Pendiente	Confirmar Eliminar
Toby	Rosa Medina	Vacunacion	16:30	Confirmada	Confirmar Eliminar
Nala	Diego Salas	Consulta	08:45	Pendiente	Confirmar Eliminar

Resumen

Citas registradas: 6

Pendientes: 3

Confirmadas: 2

Atendidas: 1

Inicio | Práctica: insumos | Panel de citas

Panel administrativo: citas

Agregar cita

Mascota

Dueño

Consulta

--:--

Guardar

Buscar por mascota

Mascota	Dueño	Servicio	Hora	Estado	Acciones
Firulais	Marta Quispe	Vacunacion	09:00	Atendida	Confirmar Eliminar
Michi	Jorge Paredes	Consulta	10:30	Confirmada	Confirmar Eliminar
Rocky	Carla Rios	Cirugia	15:00	Confirmada	Confirmar Eliminar
Luna	Pedro Soto	Consulta	11:15	Pendiente	Confirmar Eliminar
Toby	Rosa Medina	Vacunacion	16:30	Confirmada	Confirmar Eliminar
Nala	Diego Salas	Consulta	08:45	Pendiente	Confirmar Eliminar

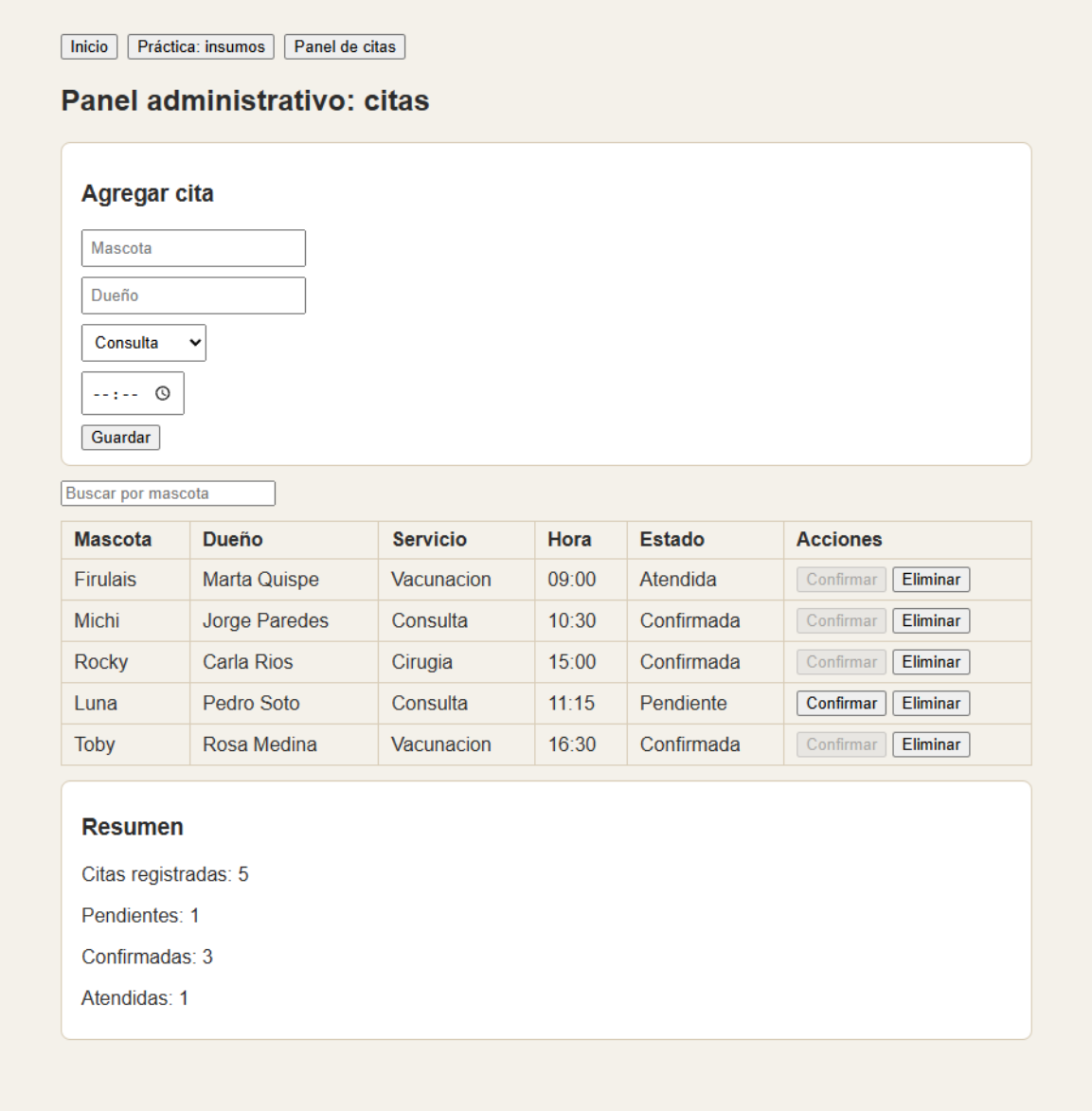
Resumen

Citas registradas: 6

Pendientes: 2

Confirmadas: 3

Atendidas: 1



Ejercicio 12 — Formulario de alta que avisa al padre

Cree un formulario hijo, con campos controlados, para dar de alta un registro de su entidad principal. Valide al menos dos reglas (las mismas que aplica su API de la Semana 8, por ejemplo, un campo obligatorio y un número mayor que cero) y muestre el mensaje de error. Si es válido, el formulario debe enviar el registro al padre con una función onGuardar, y el padre debe agregarlo con un arreglo nuevo; luego los campos quedan vacíos. Documente un alta válida y una inválida.

Respuesta:

CitaForm es un formulario hijo con campos controlados para mascota, dueño, servicio y hora. Valida dos reglas: la mascota y el dueño son obligatorios, y la hora debe estar entre las 08:00 y las 18:00. Si hay un error muestra el mensaje; si es válido envía la cita al padre con onGuardar, el padre la agrega con un arreglo nuevo y los campos quedan vacíos. Inválido: formulario vacío, y luego una hora de las 07:00. Válido: Kira, de Ana Lopez, a las 10:00, que se agrega como pendiente.

```jsx
// components/CitaForm.jsx
import { useState } from 'react'

const formularioVacio = { mascota: '', dueno: '', servicio: 'Consulta', hora: '' }

// Formulario controlado: valida y entrega la cita nueva al padre con onGuardar
function CitaForm({ onGuardar }) {
  const [form, setForm] = useState(formularioVacio)
  const [error, setError] = useState('')

  function manejarCambio(e) {
    setForm({ ...form, [e.target.name]: e.target.value })
  }

  function manejarEnvio(e) {
    e.preventDefault()
    if (form.mascota.trim() === '' || form.dueno.trim() === '') {
      setError('La mascota y el dueño son obligatorios.')
      return
    }
    if (form.hora === '' || form.hora < '08:00' || form.hora > '18:00') {
      setError('La hora debe estar entre 08:00 y 18:00.')
      return
    }
    onGuardar({
      mascota: form.mascota.trim(),
      dueno: form.dueno.trim(),
      servicio: form.servicio,
      hora: form.hora,
    })
    setForm(formularioVacio)
    setError('')
  }

  return (
    <form onSubmit={manejarEnvio} className="formulario">
      <h3>Agregar cita</h3>
      <input name="mascota" placeholder="Mascota" value={form.mascota} onChange={manejarCambio} />
      <input name="dueno" placeholder="Dueño" value={form.dueno} onChange={manejarCambio} />
      <select name="servicio" value={form.servicio} onChange={manejarCambio}>
        <option>Consulta</option>
        <option>Vacunacion</option>
        <option>Cirugia</option>
      </select>
      <input name="hora" type="time" value={form.hora} onChange={manejarCambio} />
      <button type="submit">Guardar</button>
      {error && <p className="error">{error}</p>}
    </form>
  )
}

export default CitaForm
```

Inicio | Práctica: insumos | Panel de citas

Panel administrativo: citas

Agregar cita

Mascota

Dueño

Consulta

--:--

Guardar

La mascota y el dueño son obligatorios.

Buscar por mascota

Mascota	Dueño	Servicio	Hora	Estado	Acciones
Firulais	Marta Quispe	Vacunacion	09:00	Atendida	Confirmar Eliminar
Michi	Jorge Paredes	Consulta	10:30	Confirmada	Confirmar Eliminar
Rocky	Carla Rios	Cirugia	15:00	Confirmada	Confirmar Eliminar
Luna	Pedro Soto	Consulta	11:15	Pendiente	Confirmar Eliminar
Toby	Rosa Medina	Vacunacion	16:30	Confirmada	Confirmar Eliminar

Resumen

Citas registradas: 5

Pendientes: 1

Confirmadas: 3

Atendidas: 1

Inicio | Práctica: insumos | Panel de citas

Panel administrativo: citas

Agregar cita

Kira

Ana Lopez

Consulta

07:00

Guardar

La hora debe estar entre 08:00 y 18:00.

Buscar por mascota

Mascota	Dueño	Servicio	Hora	Estado	Acciones
Firulais	Marta Quispe	Vacunacion	09:00	Atendida	Confirmar Eliminar
Michi	Jorge Paredes	Consulta	10:30	Confirmada	Confirmar Eliminar
Rocky	Carla Rios	Cirugia	15:00	Confirmada	Confirmar Eliminar
Luna	Pedro Soto	Consulta	11:15	Pendiente	Confirmar Eliminar
Toby	Rosa Medina	Vacunacion	16:30	Confirmada	Confirmar Eliminar

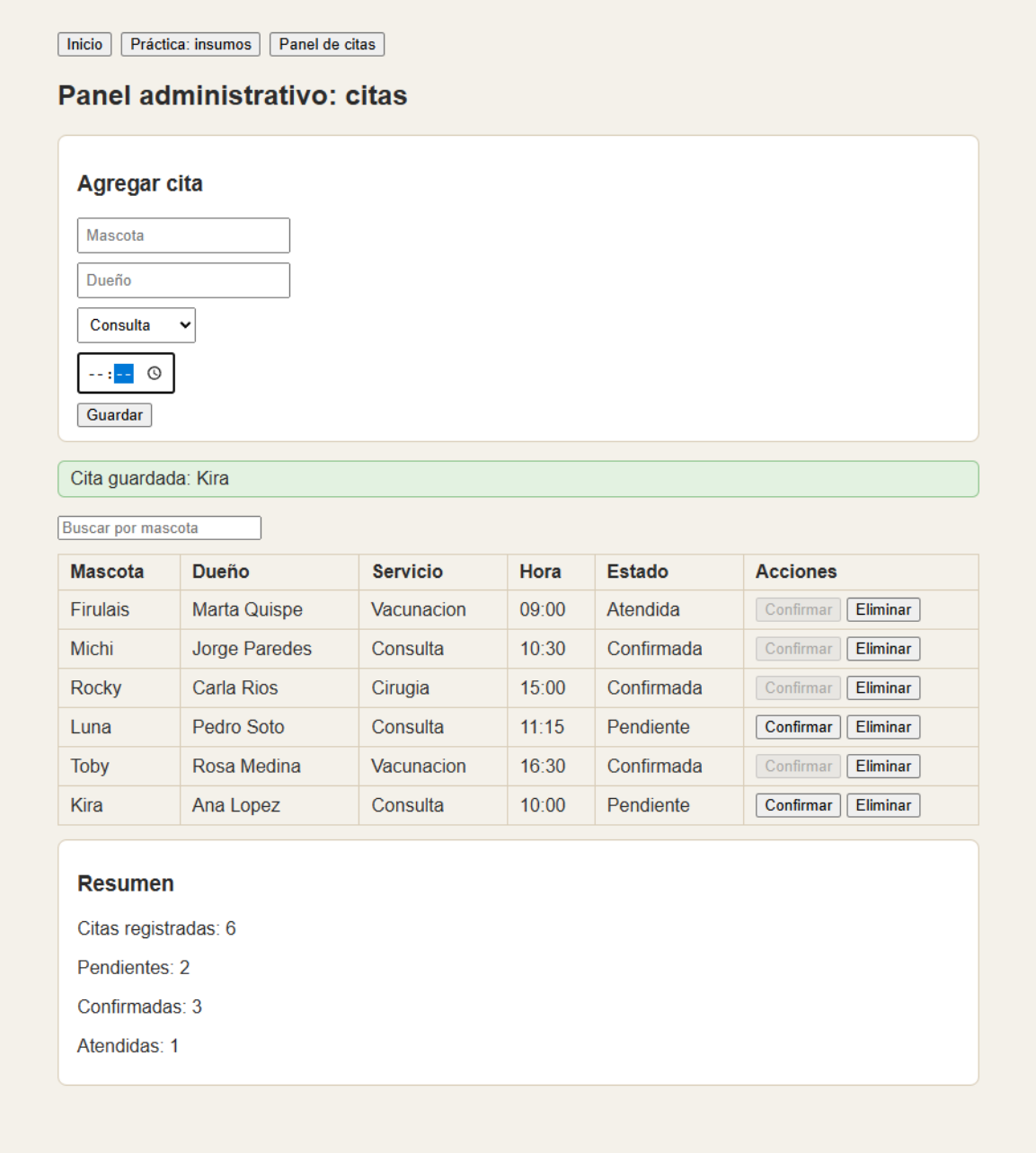
Resumen

Citas registradas: 5

Pendientes: 1

Confirmadas: 3

Atendidas: 1



Ejercicio 13 — Resumen, buscador y useEffect en el panel

Agregue un componente hermano del listado que muestre un resumen con datos derivados de su entidad (total, conteo por estado o promedio), calculados sin useState. Agregue un buscador o filtro controlado por el padre que actúe sobre el listado, con el mensaje correspondiente cuando no haya resultados. Agregue un useEffect con arreglo de dependencias (por ejemplo, el título de la pestaña con la cantidad de registros, o un aviso que desaparece a los pocos segundos con su limpieza). Documente el código y la pantalla.

Respuesta:

ResumenCitas es hermano del listado y muestra el total de citas y el conteo por estado, calculados con los datos recibidos y sin estado propio. El buscador por mascota está controlado por el padre y filtra el listado; si no hay resultados muestra No hay citas que coincidan. Además hay un efecto con arreglo de dependencias que cambia el título de la pestaña según la cantidad de citas, y un aviso que desaparece a los 3 segundos con su limpieza. Búsquedas: lu muestra a Luna; zzz muestra el mensaje.

```jsx
// components/ResumenCitas.jsx
// Resumen hermano del listado: todo se calcula con las citas recibidas, sin estado propio
function ResumenCitas({ citas }) {
  const pendientes = citas.filter((c) => c.estado === 'Pendiente').length
  const confirmadas = citas.filter((c) => c.estado === 'Confirmada').length
  const atendidas = citas.filter((c) => c.estado === 'Atendida').length

  return (
    <div className="panel-resumen">
      <h3>Resumen</h3>
      <p>Citas registradas: {citas.length}</p>
      <p>Pendientes: {pendientes}</p>
      <p>Confirmadas: {confirmadas}</p>
      <p>Atendidas: {atendidas}</p>
    </div>
  )
}

export default ResumenCitas

// components/BuscadorCitas.jsx
// Filtro de texto controlado: el valor y el cambio los maneja el padre
function BuscadorCitas({ valor, onCambiar }) {
  return (
    <input
      type="text"
      placeholder="Buscar por mascota"
      value={valor}
      onChange={(e) => onCambiar(e.target.value)}
    />
  )
}

export default BuscadorCitas

// components/PanelCitas.jsx
import { useEffect, useState } from 'react'
import { citasPanel } from '../datos.js'
import CitaListado from './CitaListado.jsx'
import CitaForm from './CitaForm.jsx'
import ResumenCitas from './ResumenCitas.jsx'
import BuscadorCitas from './BuscadorCitas.jsx'
import Aviso from './Aviso.jsx'

// Componente padre del panel: aquí vive el estado de las citas
function PanelCitas() {
  const [citas, setCitas] = useState(citasPanel)
  const [busqueda, setBusqueda] = useState('')
  const [aviso, setAviso] = useState('')

  const visibles = citas.filter((c) =>
    c.mascota.toLowerCase().includes(busqueda.toLowerCase())
  )

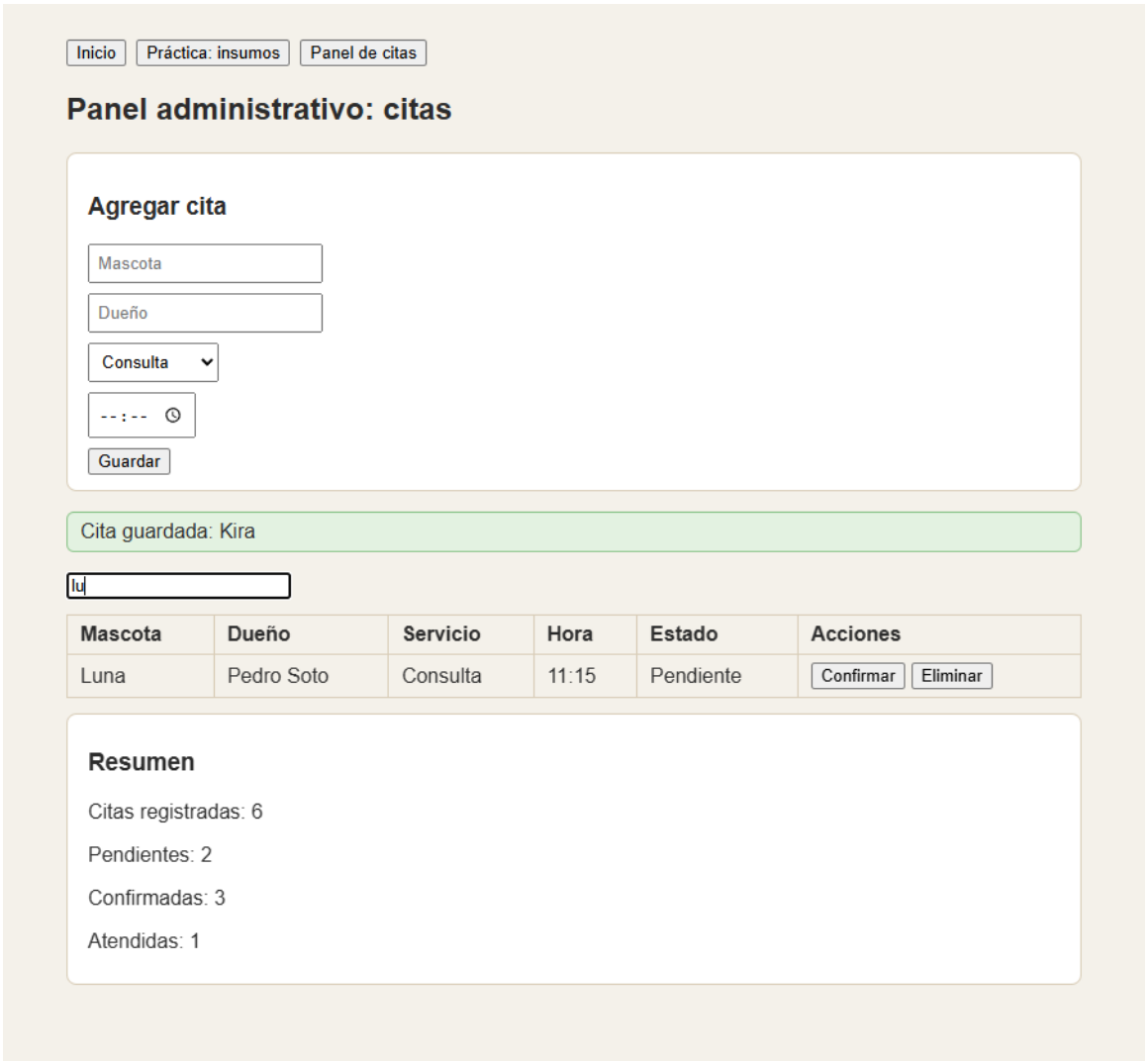
  // El título de la pestaña sigue la cantidad de citas
  useEffect(() => {
    document.title = `Panel de citas (${citas.length})`
  }, [citas.length])

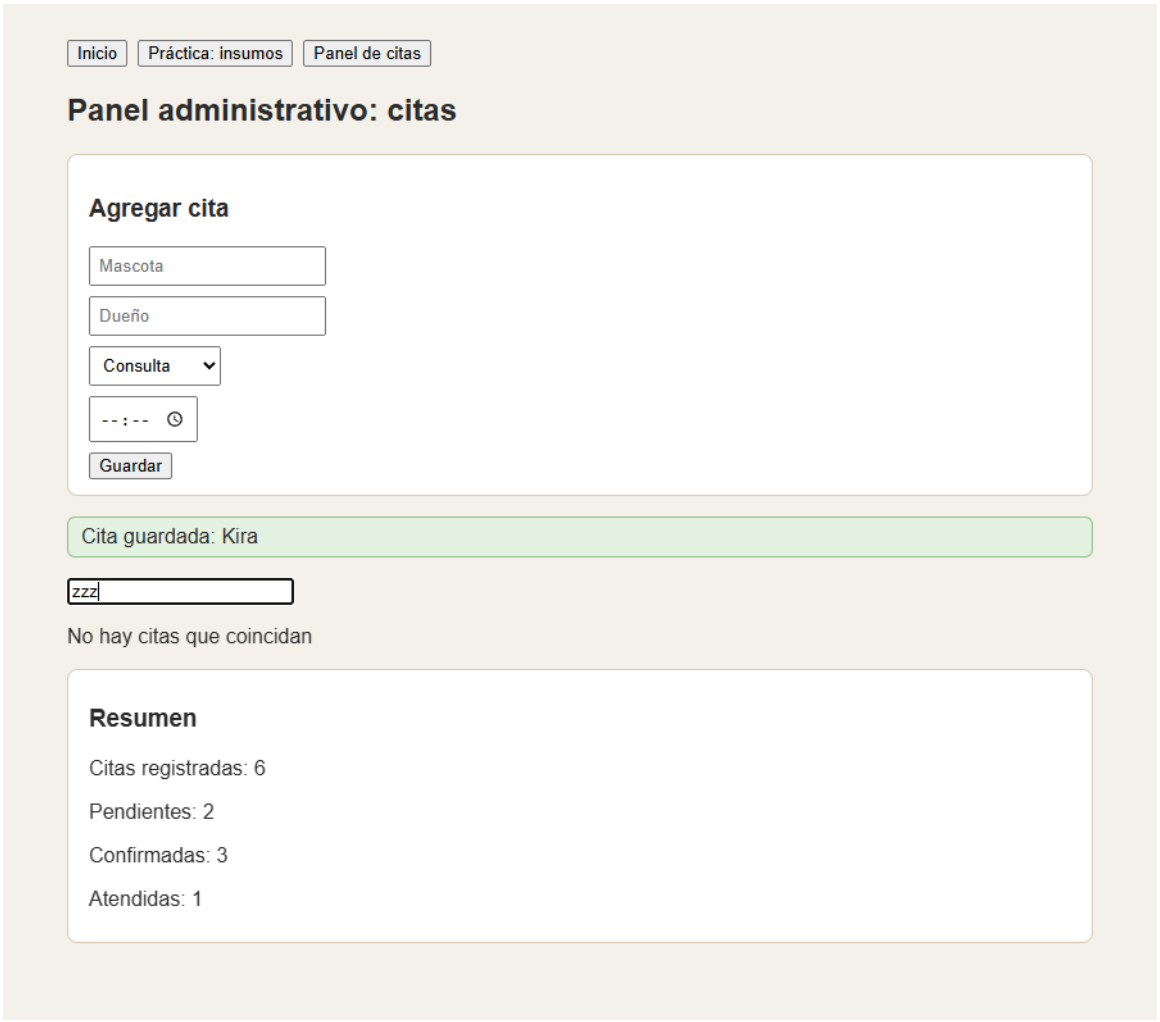
  function guardarCita(datos) {
    const nueva = { id: Math.max(0, ...citas.map((c) => c.id)) + 1, ...datos, estado: 'Pendiente' }
    setCitas([...citas, nueva])
    setAviso(`Cita guardada: ${nueva.mascota}`)
  }

  function eliminarCita(id) {
    setCitas(citas.filter((c) => c.id !== id))
  }

  function confirmarCita(id) {
    setCitas(citas.map((c) => (c.id === id ? { ...c, estado: 'Confirmada' } : c)))
  }

  return (
    <div className="seccion-vet">
      <h2>Panel administrativo: citas</h2>
      <CitaForm onGuardar={guardarCita} />
      <Aviso mensaje={aviso} onTerminar={() => setAviso('')} />
      <BuscadorCitas valor={busqueda} onCambiar={setBusqueda} />
      <CitaListado citas={visibles} onConfirmar={confirmarCita} onEliminar={eliminarCita} />
```


Inicio
Práctica: insumos
Panel de citas
Panel administrativo: citas
Agregar cita
Mascota
Dueño
Consulta
--:--
Guardar
Cita guardada: Kira
lu
Mascota
Dueño
Servicio
Hora
Estado
Acciones
Luna
Pedro Soto
Consulta
11:15
Pendiente
Confirmar
Eliminar
Resumen
Citas registradas: 6
Pendientes: 2
Confirmadas: 3
Atendidas: 1




```
Título de la pestaña del navegador (cambia con la cantidad de citas)
Panel de citas (6)
```

Ejercicio 14 — Verificar y publicar en GitHub

Verifique que la consola del navegador no muestra advertencias (por ejemplo, de key), ejecute npm run lint y npm run build, y documente la salida. Compruebe que node_modules y dist no están en el repositorio. Redacte un párrafo que explique qué datos bajan por props en su panel, qué avisos suben por funciones y por qué su estado vive en ese componente padre. Actualice el README.md con una descripción de la pantalla del panel y haga commit y push. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

Respuesta:

La consola del navegador no muestra advertencias, npm run lint no reporta errores y npm run build termina correctamente. Las carpetas node_modules y dist están ignoradas por el repositorio y no se suben. En mi panel bajan por props las citas hacia el listado, las filas y el resumen, el texto hacia el buscador y el mensaje hacia el aviso; suben por funciones los avisos de confirmar, eliminar, guardar una cita nueva y cambiar la búsqueda. El estado vive en el componente padre PanelCitas porque es el único que ve a todos los hermanos: el listado, el formulario y el resumen necesitan las mismas citas, y así cualquier cambio se refleja a la vez en todas partes. Repositorio: https://github.com/RXDG2908/DesAplEmp-S02

```
Consola del navegador tras usar el panel
Sin errores ni advertencias.
```

```
PS ...\frontend> npm run lint

> frontend@0.0.0 lint
> eslint .
```

```
PS ...\frontend> npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.5 building client environment for production...
transforming...
✓ 43 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  0.47 kB | gzip:  0.30 kB
dist/assets/index-xrUpEBpY.css   1.27 kB | gzip:  0.47 kB
dist/assets/index-yf61NHLG.js  236.51 kB | gzip: 73.25 kB

✓ built in 274ms
```

```
PS ...\frontend> git check-ignore -v node_modules dist
frontend/.gitignore:10:node_modules     node_modules
frontend/.gitignore:11:dist     dist
Archivos de node_modules o dist dentro del repositorio: 0
```

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de los componentes, la página en el navegador y la salida de npm run lint y npm run build).
- Repositorio en GitHub actualizado, con la carpeta frontend (incluida src/practica y los componentes del panel), el README.md y el código correspondiente, sin node_modules ni dist.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Repositorio: https://github.com/RXDG2908/DesAplEmp-S02

Conclusiones:

Los datos bajan del padre al hijo por props y los avisos suben del hijo al padre con funciones recibidas por props.
Cuando dos componentes hermanos necesitan el mismo dato, el estado se eleva al padre que los contiene.
Lo que se puede calcular a partir del estado, como el total o el conteo por estado, no necesita su propio estado.
El estado siempre se cambia con arreglos nuevos, usando map, filter o spread, nunca con push ni splice.

El efecto con arreglo de dependencias corre solo cuando cambian esos datos, y la limpieza cancela el temporizador del aviso.
En el panel de la veterinaria el listado, el formulario, el buscador y el resumen se comunican a través del componente padre PanelCitas.
Los datos del panel todavía están escritos a mano; lo que sigue es consumir la API de Django.